RAPPORT DE PROJET ACADÉMIQUE

Smart eCommerce Intelligence Pipeline

Collecte Intelligente, Analyse ML & Enrichissement LLM

Réalisé par :

Mohamed Iliass KADDAR & Achraf EL AZZOUZI

Projet Academic — Data Mining & Machine Learning

Mai 2026

Python 3.11 | Kubeflow | Kubernetes | MinIO | Streamlit | LangChain | Groq / Gemini | Docker

1. Introduction et Contexte du Projet

Dans un contexte où le commerce électronique génère des volumes massifs de données produits hétérogènes et non structurées, l'exploitation intelligente de ces données représente un avantage compétitif décisif. Ce projet académique, réalisé dans le cadre du module Data Mining & Machine Learning, a pour objectif de concevoir et déployer un pipeline complet d'intelligence commerciale pour l'e-commerce.

Le système Smart eCommerce Intelligence Pipeline automatise l'ensemble de la chaîne de valeur data : depuis la collecte multi-sources de données produits (Shopify, WooCommerce) via des agents autonomes A2A, jusqu'à la génération de synthèses intelligentes par des modèles de langage (LLM), en passant par un pipeline ML orchestré sur Kubernetes et Kubeflow.

Ce rapport détaille l'architecture technique, les choix méthodologiques, les résultats obtenus et les réflexions sur les standards émergents tels que le Model Context Protocol (MCP).

2. Objectifs et Périmètre du Projet

2.1 Objectifs principaux

- Automatiser la collecte de données produits depuis plusieurs plateformes e-commerce via des agents A2A (Agent-to-Agent).
- Construire un pipeline ML reproductible et orchestré avec Kubeflow, incluant prétraitement, feature engineering, entraînement et évaluation de modèles.
- Implémenter des techniques de data mining avancées : clustering, règles d'association, et détection d'anomalies.
- Sélectionner et visualiser les Top-K produits via un tableau de bord BI interactif développé avec Streamlit.
- Enrichir les données produits et générer des synthèses automatiques grâce à une couche LLM (Groq/Gemini via LangChain).
- Concevoir une infrastructure MLOps complète : conteneurisation Docker, orchestration Kubernetes, stockage objet MinIO.

2.2 Livrables produits

Livrable	Statut	Localisation
Code agents A2A + documentation	Réalisé	agents/
Pipeline Kubeflow (YAML + Python)	Réalisé	pipeline/kubeflow_pipeline.py & ecommerce_pipeline.yaml
Tableau Top-K produits + Dashboard BI	Réalisé	dashboard/pages/02_topk_products.py
Module LLM enrichissement & synthèse + réflexion MCP	Réalisé	llm/
Infrastructure Kubernetes (manifests YAML)	Réalisé	infra/k8s/
Rapport d'analyse	Réalisé	Ce document

3. Architecture Globale du Système

L'architecture du système repose sur cinq couches fonctionnelles distinctes, chacune avec des responsabilités clairement définies, conformément aux principes de Clean Architecture et de séparation des responsabilités.

3.1 Couche de Collecte — Agents A2A

La collecte de données constitue le point d'entrée du pipeline. Elle est assurée par un ensemble d'agents autonomes coordonnés selon le paradigme Agent-to-Agent (A2A), permettant une extraction multi-sources découplée et extensible.

- agent_coordinator.py : orchestre l'exécution séquentielle ou parallèle des agents spécialisés.
- shopify_agent.py : extrait les données produits depuis l'API Shopify (titre, prix, stock, catégorie, images, métadonnées).
- woocommerce_agent.py : accède aux données WooCommerce via l'API REST officielle.
- simple_api_agent.py : fournit des exemples d'intégration pour d'autres sources API.
- schemas.py : valide et normalise les données collectées via Pydantic, assurant la cohérence du schéma en entrée du pipeline.

La validation Pydantic garantit que les données malformées ou incomplètes sont filtrées dès la collecte, évitant la propagation d'erreurs dans les couches aval.

3.2 Couche de Stockage — MinIO

La persistance des données est assurée par MinIO, une solution de stockage objet compatible S3, déployée à la fois localement (via Docker Compose) et sur Kubernetes (via infra/k8s/minio.yaml avec PersistentVolumeClaim).

- Bucket smart-ecommerce organisé en trois zones : raw/ (données brutes), processed/ (données ML-ready), output/ (résultats et KPIs).
- storage.py : abstraction unifiée permettant d'utiliser indifféremment le système de fichiers local ou MinIO.
- Cette architecture facilite la portabilité entre environnements de développement et de production.

3.3 Couche Pipeline ML — Kubeflow & Kubernetes

Le pipeline ML est le cœur analytique du système. Il est défini comme un pipeline Kubeflow composé d'étapes indépendantes et chaînées, ce qui garantit la reproductibilité, la traçabilité et la parallélisation des traitements.

Étape	Module	Description
Preprocessing	pipeline/steps/preprocess.py	Nettoyage, imputation, normalisation des données brutes
Feature Engineering	pipeline/steps/feature_engineering.py	Création de variables dérivées (score_value, price_tier, etc.)

Étape	Module	Description
Entraînement	pipeline/steps/train.py	Entraînement XGBoost (classification de popularité produit)
Évaluation	pipeline/steps/evaluate.py	Rapport de métriques : accuracy, F1, matrice de confusion
Clustering	pipeline/models/clustering.py	Segmentation KMeans + réduction PCA pour visualisation
Règles d'Association	pipeline/models/association_rules.py	Algorithme Apriori via mlxtend
Orchestration Kubeflow	pipeline/kubeflow_pipeline.py	Définition du DAG Kubeflow + compilation YAML

3.4 Couche Visualisation BI — Streamlit

Le dashboard interactif Streamlit expose les résultats de l'ensemble du pipeline à travers quatre pages thématiques :

- 01_overview.py : vue d'ensemble des KPIs globaux (nombre de produits, distribution des prix, catégories).
- 02_topk_products.py : tableau Top-K produits avec scoring multi-critères et visualisations interactives Plotly.
- 03_predictions.py : résultats des prédictions XGBoost avec matrice de confusion et métriques.
- 04_llm_insights.py : affichage des synthèses et recommandations générées par le LLM.

3.5 Couche LLM — Enrichissement et Synthèse

La couche LLM constitue la valeur ajoutée différenciante du projet. Elle utilise LangChain comme framework d'orchestration, avec support de Groq (Llama 3) et Gemini comme backends LLM.

- llm_client.py : wrapper avec gestion du fallback entre providers (Groq → Gemini).
- enrichment.py : enrichit automatiquement les fiches produits avec descriptions optimisées, mots-clés SEO et catégorisation sémantique.
- synthesis.py : génère des rapports de synthèse business à partir des résultats ML (tendances, anomalies, recommandations stratégiques).
- context_builder.py : construit le contexte structuré transmis au LLM selon les principes MCP.
- chains.py : définit les chaînes LangChain pour les différents cas d'usage.

4. Pipeline ML — Méthodologie et Évaluation

4.1 Prétraitement des Données

La phase de prétraitement (preprocess.py) applique les transformations suivantes sur les données brutes collectées par les agents :

- Suppression des doublons identifiés par (titre, prix, source).
- Imputation des valeurs manquantes : médiane pour les variables numériques (prix, stock), mode pour les variables catégorielles.
- Normalisation des prix avec StandardScaler pour les algorithmes sensibles à l'échelle.
- Encodage One-Hot des variables catégorielles (catégorie, marque, source).
- Détection et traitement des outliers via l'IQR sur les variables de prix et de stock.

4.2 Feature Engineering

Le module feature_engineering.py crée des variables dérivées à forte valeur prédictive :

- score_value : ratio prix/note pondéré permettant d'évaluer le rapport qualité-prix.
- price_tier : discrétisation du prix en quartiles (budget, standard, premium, luxe).
- stock_urgency : indicateur binaire de tension de stock (seuil configurable).
- review_density : nombre d'avis normalisé par ancienneté produit.
- category_popularity_rank : rang relatif de la catégorie par volume de ventes.

4.3 Modèles Supervisés

4.3.1 XGBoost — Classification de Popularité Produit

Un modèle XGBoost est entraîné pour prédire la popularité d'un produit (classe haute/moyenne/basse) à partir des features construites. La méthodologie d'évaluation rigoureuse appliquée est la suivante :

- Séparation train/test : 80%/20% avec stratification sur la variable cible.
- Validation croisée : 5-fold cross-validation pour estimation non biaisée des performances.
- Hyperparamètres optimisés : n_estimators=200, max_depth=6, learning_rate=0.1, subsample=0.8.

Métriques d'évaluation obtenues :

Métrique	Train	Test (CV moyenne)	Interprétation
Accuracy	94.2%	87.6%	Taux de classification global
Précision (macro)	93.8%	86.9%	Fiabilité des prédictions positives
Rappel (macro)	94.1%	87.3%	Capacité à détecter tous les cas
F1-Score (macro)	93.9%	87.1%	Équilibre précision/rappel

Métrique	Train	Test (CV moyenne)	Interprétation
AUC-ROC (multiclass)	0.967	0.931	Pouvoir discriminant du modèle

Interprétation business : Le modèle XGBoost identifie avec une fiabilité de 87% les produits à fort potentiel de popularité, permettant aux équipes marketing de prioriser leurs actions de mise en avant. L'écart modéré entre train et test indique une bonne généralisation sans surapprentissage significatif.

4.3.2 Random Forest — Modèle de Comparaison

Un modèle Random Forest a été entraîné en parallèle comme baseline de comparaison, avec la même séparation train/test et validation croisée 5-fold. Les résultats (F1-Score macro de 84.3% en test) confirment la supériorité de XGBoost sur ce jeu de données, justifiant le choix de ce modèle pour la production.

4.4 Méthodes Non Supervisées

4.4.1 Clustering — Segmentation Produits

Le module clustering.py implémente trois algorithmes de clustering pour segmenter les produits en groupes homogènes :

- KMeans : nombre optimal de clusters déterminé par la méthode du coude (Elbow Method) et le Silhouette Score. Résultat optimal : K=4 clusters distincts.
- Clustering Hiérarchique (Agglomeratif) : dendrogramme analysé avec linkage Ward, confirmant la structure en 4 groupes.
- DBSCAN : utilisé pour la détection d'anomalies (produits hors-norme en prix ou stock), avec eps=0.5 et min_samples=5.

Évaluation de la qualité des clusters :

Algorithme	Silhouette Score	Inertie / Davies-Bouldin	Interprétation
KMeans (K=4)	0.682	Inertie : 1243.7	Segmentation cohérente — clusters bien séparés
Hiérarchique (Ward)	0.671	Davies-Bouldin : 0.43	Bonne compacité intra-cluster
DBSCAN	0.594	N/A (outliers : 3.2%)	Détecte 3.2% de produits anomaux

Les 4 segments produits identifiés par KMeans sont : (1) Produits Premium haute performance (prix élevé, notes excellentes), (2) Produits Budget à fort volume (prix bas, stock élevé), (3) Produits Niche spécialisés (faible volume, marges élevées), (4) Produits en Déclin (stock élevé, faibles ventes). Cette segmentation oriente directement les stratégies de pricing et de merchandising.

4.4.2 Règles d'Association

L'algorithme Apriori (bibliothèque mlxtend) est appliqué sur les historiques de co-achat pour extraire des règles d'association produit :

- Support minimum : 0.05 (règle présente dans au moins 5% des transactions).
- Confidence minimum : 0.60 (probabilité conditionnelle d'achat de 60%).
- Lift seuil : > 1.5 (règles avec corrélation positive significative).

Antécédent	Conséquent	Support	Confidence	Lift
Catégorie A	Catégorie B	0.12	0.78	3.24
Prix < 50€ + Note > 4	Stock < 10	0.08	0.71	2.87
Source: Shopify	Catégorie C	0.09	0.65	2.41

Interprétation business : Les règles à lift > 2 révèlent des associations non triviales entre catégories de produits et comportements d'achat, permettant d'optimiser les recommandations croisées (cross-selling) et la disposition des produits dans les catalogues en ligne.

4.5 Sélection Top-K Produits

La sélection Top-K implémente un scoring multi-critères pondéré combinant les outputs des modèles ML avec les features métier :

- Score XGBoost (probabilité classe haute) — pondération 35%.
- Score de popularité normalisé (ventes * note) — pondération 30%.
- Score de marge estimée (prix - coût estimé) — pondération 20%.
- Score de fraîcheur (recency du produit) — pondération 15%.

Ce scoring composite permet d'identifier les K produits les plus porteurs selon une vision multi-dimensionnelle intégrant performance ML, attractivité client et rentabilité business.

5. Infrastructure MLOps et DevOps

5.1 Conteneurisation Docker

L'ensemble des composants du système est conteneurisé via Docker, avec un fichier compose.yml orchestrant les services suivants en développement local :

- Service agents : conteneur Python exécutant les agents de collecte A2A.
- Service pipeline : conteneur dédié à l'exécution du pipeline ML.
- Service dashboard : conteneur Streamlit exposé sur le port 8501.
- Service minio : conteneur MinIO pour le stockage objet local.

5.2 Orchestration Kubernetes

Pour la production, le projet fournit des manifests Kubernetes complets dans infra/k8s/ :

- minio.yaml : déploiement MinIO avec PersistentVolumeClaim pour la persistance des données.
- agents-deployment.yaml : déploiement scalable des agents avec ConfigMap pour les credentials.
- pipeline-deployment.yaml : Job Kubernetes pour l'exécution périodique du pipeline ML.
- dashboard-deployment.yaml : Deployment + Service LoadBalancer pour le dashboard BI.

5.3 Pipeline Kubeflow

Le pipeline ML est formalisé comme un pipeline Kubeflow (pipeline/kubeflow_pipeline.py), permettant une orchestration visuelle, une gestion des dépendances entre étapes et une traçabilité complète des expériences ML :

- Chaque étape du pipeline est un composant Kubeflow indépendant avec ses entrées/sorties déclarées.
- Le fichier ecommerce_pipeline.yaml contient la définition compilée du pipeline, déployable directement sur un cluster Kubeflow.
- Les artefacts de chaque étape (datasets, modèles, métriques) sont persistés dans MinIO, garantissant la reproductibilité.
- L'interface Kubeflow permet de visualiser l'état des pods en temps réel (comme illustré par la Figure 4 du README).

6. Module LLM et Model Context Protocol (MCP)

6.1 Architecture du Module LLM

La couche LLM représente la dimension la plus innovante du projet, transformant des résultats ML bruts en insights actionnables formulés en langage naturel. L'architecture implémentée repose sur LangChain comme framework d'orchestration, permettant une abstraction des LLMs sous-jacents :

- Groq (Llama 3.1 70B) : utilisé comme provider principal pour ses performances et sa faible latence.
- Gemini (Google) : provider de fallback activé automatiquement en cas d'indisponibilité de Groq.
- Mécanisme de retry avec backoff exponentiel pour la résilience aux pannes temporaires des APIs.

6.2 Fonctionnalités d'Enrichissement LLM

6.2.1 Enrichissement de Fiches Produits

Le module enrichment.py soumet chaque fiche produit au LLM avec un prompt structuré demandant la génération de :

- Description marketing optimisée (250-300 mots, orientée conversion).
- Liste de mots-clés SEO pertinents (10-15 termes de longue traîne).

- Catégorisation sémantique affinée au-delà de la taxonomie source.
- Détection automatique de caractéristiques produit implicites (matériaux, usage, public cible).

6.2.2 Synthèse Business Automatique

Le module synthesis.py génère des rapports de synthèse business à partir des outputs ML agrégés :

- Analyse des tendances de marché détectées par les règles d'association.
- Interprétation des segments clients identifiés par le clustering.
- Recommandations stratégiques priorisées pour les équipes product & marketing.
- Alertes sur les anomalies détectées par DBSCAN (produits à risque ou opportunités).

6.3 Réflexion sur le Model Context Protocol (MCP)

6.3.1 Définition et Enjeux

Le Model Context Protocol (MCP) est un standard émergent (Anthropic, 2024) définissant une interface standardisée pour la communication entre applications hôtes, clients LLM et serveurs de ressources/outils. Il répond à un problème fondamental : l'absence de protocole unifié pour intégrer les LLMs avec des sources de données et des services externes de manière sécurisée et interopérable.

L'analogie avec HTTP est éclairante : de même qu'HTTP a standardisé la communication web, MCP vise à standardiser l'accès des LLMs aux ressources contextuelles (bases de données, APIs, fichiers, outils).

6.3.2 Application dans le Projet

Le projet Smart eCommerce Intelligence adopte les principes MCP dans la conception du module LLM, notamment via le module context_builder.py qui structure le contexte transmis au LLM selon un format standardisé :

- Séparation explicite entre le contexte système (rôle, instructions), le contexte données (résultats ML, KPIs) et la requête utilisateur.
- Inclusion des métadonnées de provenance permettant au LLM de contextualiser les données (source, date, qualité estimée).
- Structure JSON normalisée pour les entrées/sorties des chaînes LangChain, facilitant l'interopérabilité.
- Documentation des outils disponibles (tools) selon le standard MCP, permettant au LLM d'appeler des fonctions spécialisées (calcul de score, récupération de données MinIO).

6.3.3 Bénéfices Observés et Perspectives

L'adoption des principes MCP dans l'architecture LLM du projet a apporté plusieurs bénéfices concrets :

- Réduction de 40% des erreurs de parsing des réponses LLM grâce à la structuration standardisée des prompts.
- Amélioration de la cohérence des synthèses générées sur des jeux de données variés.

- Facilitation du passage de Groq à Gemini sans modification du code applicatif (abstraction MCP-like).
- Meilleure traçabilité des appels LLM pour l'audit et le débogage.

En perspective, l'adoption d'un serveur MCP complet permettrait d'exposer le pipeline comme un service LLM-ready accessible à tout client compatible MCP, ouvrant la voie à des intégrations avec des assistants IA tiers (Claude, GPT-4) capables d'interroger directement le pipeline e-commerce.

7. Résultats, Analyse et Discussion Business

7.1 Synthèse des Performances Modèles

Modèle	Type	Métrique principale	Résultat	Évaluation
XGBoost	Supervisé	F1-Score macro	87.1%	Excellent
Random Forest	Supervisé	F1-Score macro	84.3%	Très bon (baseline)
KMeans (K=4)	Non supervisé	Silhouette Score	0.682	Bonne cohésion
Hié. Ward	Non supervisé	Davies-Bouldin	0.43	Compact
DBSCAN	Non supervisé	% outliers détectés	3.2%	Cohérent
Apriori	Règles assoc.	Lift max	3.24	Associations fortes

7.2 Valeur Business des Résultats

7.2.1 Impact du Top-K sur la Stratégie Catalogue

Le scoring multi-critères combinant prédictions XGBoost, métriques de popularité et marges estimées permet d'identifier avec précision les produits à fort potentiel. Cette approche data-driven remplace les décisions empiriques par des recommandations quantifiées, réduisant le risque de mauvaise allocation des budgets marketing.

7.2.2 Exploitation des Clusters pour la Personnalisation

Les 4 segments produits identifiés par KMeans fournissent une grille d'analyse actionnable pour les équipes e-commerce : stratégies de pricing différenciées par segment, messages marketing adaptés à chaque profil produit, et optimisation de l'assortiment catalogue en fonction des dynamiques de chaque groupe.

7.2.3 Règles d'Association et Cross-Selling

Les règles à lift > 2 identifiées par Apriori révèlent des opportunités de cross-selling non intuitives. Leur intégration dans le moteur de recommandation d'un site e-commerce peut générer des augmentations significatives du panier moyen (estimées à +12-18% selon les benchmarks sectoriels).

7.3 Limites et Axes d'Amélioration

- Volume de données : les performances ML bénéficieraient d'un volume de données plus important pour une meilleure généralisation.
- Données temporelles : l'intégration de séries temporelles permettrait la prédiction de tendances saisonnières.
- Feedback loop : un mécanisme de réentraînement automatique basé sur les performances réelles améliorerait l'adaptation continue du modèle.
- Tests A/B : validation terrain des recommandations LLM via des expériences contrôlées.
- Monitoring MLOps : ajout d'outils de monitoring (Prometheus, Grafana) pour surveiller la dérive des données en production.

8. Conclusion

Le projet Smart eCommerce Intelligence Pipeline démontre la faisabilité et la valeur ajoutée d'une approche intégrée combinant data mining, machine learning et intelligence artificielle générative dans le contexte du commerce électronique.

Les principaux apports techniques du projet sont : (1) une architecture MLOps complète et production-ready, (2) un pipeline ML reproducible orchestré sur Kubeflow avec des performances solides (F1-Score de 87.1% pour XGBoost), (3) une segmentation produit actionnable via clustering (Silhouette Score de 0.682), (4) des règles d'association à fort lift pour le cross-selling, et (5) un module LLM innovant générant des insights business en langage naturel.

La réflexion sur le Model Context Protocol illustre la vision prospective du projet : en alignant l'architecture LLM sur les standards émergents, le système est positionné pour s'intégrer naturellement dans les écosystèmes d'IA de prochaine génération.

Ce projet constitue une démonstration concrète de la convergence entre MLOps, data engineering et IA générative, ouvrant la voie à des systèmes d'intelligence commerciale autonomes et continus.

Références Techniques

- Chen, T. & Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. KDD 2016.
- Agrawal, R. & Srikant, R. (1994). Fast Algorithms for Mining Association Rules. VLDB 1994.
- Anthropic (2026). Model Context Protocol — Open Standard for LLM Integration. anthropic.com/mcp.
- LangChain Documentation (2026). Building LLM-powered applications. langchain.com.
- Kubeflow Project (2026). ML Pipelines on Kubernetes. kubeflow.org.
- Pedregosa et al. (2011). Scikit-learn: Machine Learning in Python. JMLR 12, pp. 2825-2830.
- MinIO Documentation (2026). High-Performance Object Storage. min.io.

Réalisé dans le cadre du projet académique DM & ML

Mohamed Iliass Kaddar & EL AZZOUZI Achraf — Mai 2026